МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

ЗВІТ

з лабораторної роботи №6
з дисципліни “Професійний Python”
на тему “Автоматизоване тестування Python-програм”

Виконала
студентка 3 курсу
групи ФеП-33с
Корхова Марія Сергіївна

Перевірив
професор
Свелеба Сергій Андрійович

Львів 2026

Мета роботи: Сформувати практичні навички професійного автоматизованого тестування Python програм із використанням фреймворку pytest, навчитися проєктувати unit та integration tests, створювати fixtures, використовувати параметризацію, перевіряти exceptions, ізолювати файлову систему та зовнішні залежності, застосовувати mocks і аналізувати test coverage.

Варіант 7. Система оцінювання студентів Протестувати: • створення Student, Discipline, Grade та AssessmentRecord; • валідацію оцінки; • boundary мінімальної та максимальної оцінки; • invalid grade; • обчислення середньої оцінки студента; • визначення min/max оцінки; • розрахунок успішності; • сортування студентів за середнім балом; • пошук студентів вище заданого порогу; • порожній GradeBook; • CSV import оцінок; • malformed grade record та custom exception; • JSON export і configuration порогу; • GradeRepository/Notifier через Mock із side_effect; • AsyncMock та integration import → grade calculation → report.

Опис проєкту

Розроблено систему обліку успішності, яка підтримує:

- створення та валідацію оцінок (Grade), дисциплін (Discipline) та студентів (Student);
- обчислення середньої оцінки студента;
- визначення мінімальної та максимальної оцінки по всьому журналу (GradeBook);
- розрахунок загальної успішності (відсоток оцінок, вищих за поріг);
- сортування студентів за їхнім середнім балом;
- фільтрацію студентів, чий середній бал перевищує заданий поріг;
- потоковий CSV-імпорт з обробкою пошкоджених записів (malformed records);
- JSON-експорт звітів;

- завантаження конфігурації (порогового бала) через змінні середовища;
- збереження результатів через GradeRepository та сповіщення через Notifier;
- асинхронну генерацію звітів через зовнішній AsyncReportProvider.

Структура production code

Модуль	**Призначення**
`models.py`	Моделі Student, Discipline, Grade, AssessmentRecord та логіка валідації оцінки.
`book.py`	Головний клас `GradeBook`, який агрегує записи, шукає, обчислює середнє та сортує.
`config.py`	Зчитування змінних середовища (`PASS_THRESHOLD`).
`io.py`	Читання з CSV та запис JSON звітів, обробка `DataImportError`.
`services.py`	Взаємодія з `GradeRepository`, `Notifier` та асинхронним `AsyncReportProvider`.

Структура test suite

Unit tests перевіряють окремо моделі, бізнес-логіку журналу, конфігурацію та сервіси. Integration tests перевіряють наскрізний потік даних: завантаження з CSV, розрахунок середнього бала для кожного студента, фільтрацію через конфігурацію та вивантаження сформованого звіту в JSON.

Матриця тестування

Компонент	**Normal**	**Boundary**	**Invalid/failure**
Grade	Коректні значення (85.5)	Нижня (0), верхня (100) межа	Від'ємні, більші за 100

GradeBook	Середній бал, мін/макс	Один студент, одна оцінка	Порожній GradeBook (має повертати 0.0)
Пошук/Сортування	Сортування за середнім балом	Кілька студентів з однаковим балом	Відсутні студенти, порожній список
CSV import	Коректний CSV	Один запис	Пошкоджені дані, неправильний тип (`DataImportError`)
JSON export	Серіалізація словників	Порожній список на експорт	Помилки прав запису до файлу
Configuration	Зчитування кастомного порогу	Відсутня змінна середовища (fallback на 60.0)	—
Service (Mock)	Успішне збереження і виклик Notifier	—	Помилка БД (`ConnectionError`) через `side_effect`
Async Provider	Успішне завантаження звіту	Порожня відповідь	Відсутність з'єднання

Використані fixtures

Створено fixtures у `conftest.py`:

- `sample_record` — єдиний валідний запис для базових тестів.
- `populated_book` — заповнений `GradeBook`, де є студент з кількома оцінками для тестування розрахунку середнього бала.
- `tmp_valid_csv` та `tmp_invalid_csv` — фікстури у тестових модулях, які використовують `tmp_path` для генерації тестових файлів на льоту.

Це зменшує дублювання коду і гарантує кожному тесту чистий початковий стан.

Parametrization

Parametrization використано для перевірки валідації моделі `Grade`:

- Перевірка дійсних граничних значень: `[0, 50.5, 100]`.
- Перевірка недійсних значень: `[-0.1, -50, 100.1, 200]`. Це дозволило написати дві компактні тестові функції замість семи окремих.

Перевірка exceptions

Використано `pytest.raises` для перехоплення очікуваних винятків:

- `GradeValidationError` при спробі створити `Grade(-1)` або `Grade(150)`.
- `DataImportError` при читанні malformed CSV рядка.
- `ConnectionError`, коли змодельована база даних дає збій (`side_effect`). Використовувався аргумент `match`, щоб переконатися, що текст помилки відповідає очікуваному (наприклад, `match="0 and 100"`).

Перевірка floating-point результатів

Оскільки розрахунок середнього бала або відсотка успішності повертає числа з рухомою комою (float), у тестах використано `pytest.approx`.

```
assert populated_book.average_student_grade(1) == pytest.approx(95.0)
```

Це усуває проблему нестабільності порівняння через особливості двійкового представлення дробів.

Тестування файлової системи

Фікстура `tmp_path` генерує унікальні тимчасові каталоги для кожного тесту. Усі CSV файли для імпорту та JSON файли для експорту створювалися і

записувалися лише туди. Це гарантує ізоляцію: тести не смітять у робочій директорії проєкту і не конфліктують між собою.

Тестування configuration

За допомогою `monkeypatch` ізольовано тестовано конфігураційний модуль:

- `monkeypatch.setenv("PASS_THRESHOLD", "75.5")` перевіряє, чи правильно читається нове значення.
- `monkeypatch.delenv("PASS_THRESHOLD", raising=False)` перевіряє поведінку за замовчуванням (fallback на `60.0`), якщо змінна відсутня.

Mock, return_value та side_effect

У `test_services.py` ізольовано бізнес-логіку від реальних бази даних та системи сповіщень. Замість них використано `Mock`.

- Успішний сценарій: `repo.save.return_value = True`. Перевірено через `notifier.send.assert_called_once()`.
- Сценарій відмови: `repo.save.side_effect = ConnectionError("DB down")`. Перевірено, що `GradingService` прокидає виняток далі, і `notifier.send.assert_not_called()`.

AsyncMock

Для перевірки `AsyncReportService`, який звертається до зовнішнього асинхронного API, використано `AsyncMock`. Замість реального очікування мережевої відповіді, мок миттєво повертав підготовлений словник `{"status": "success"}`. Виклик функції маркувався `@pytest.mark.asyncio`, а факт правильного виклику залежності перевірявся через `provider.fetch_report.assert_awaited_once_with(1)`.

Integration tests

Реалізовано наскрізний флоу: `import → grade calculation → report`.

1. Тимчасовий CSV файл заповнюється даними.
2. `monkeypatch` встановлює поріг.

3. Парсер зчитує дані у `GradeBook`.
4. Виконується розрахунок середнього бала для визначення студентів, які подолали поріг.
5. Формується новий звіт і записується у тимчасовий JSON.
6. Тест зчитує JSON і переконується, що дані агреговані правильно (наприклад, середній бал 95.0 для студента з оцінками 90 і 100).

Результат запуску

```
tests/unit/test_book.py ......                                          [ 25%]
tests/unit/test_config.py ..                                            [ 33%]
tests/unit/test_models.py .......                                       [ 62%]
tests/unit/test_services.py ...                                         [ 75%]
tests/integration/test_flows.py ..                                      [ 83%]

============================== 24 passed ==============================
```

Coverage report

Команда запуску:`python -m pytest -v --cov=grades --cov-branch --cov-report term-missing --cov-report=html`

Name	Stmts	Miss	Branch	BrPart	Cover
`grades/__init__.py`	0	0	0	0	100%
`grades/book.py`	33	0	12	0	100%
`grades/config.py`	3	0	0	0	100%
`grades/io.py`	17	0	4	0	100%
`grades/models.py`	17	0	2	0	100%
`grades/services.py`	21	0	2	0	100%
TOTAL	**91**	**0**	**20**	**0**	**100%**

HTML-звіт збережено у директорії `htmlcov/index.html`.

Аналіз execution paths

Усі альтернативні гілки виконання (branches) покриті на 100%. Наприклад, у `GradeBook` перевіряється розрахунок середнього значення як для колекції з даними, так і для порожнього `GradeBook` (де повертається `0.0`, щоб уникнути `ZeroDivisionError`). У функції читання CSV перевіряється як блок `try`, що генерує об'єкти, так і блок `except`, який ловить `GradeValidationError` чи `KeyError` і прокидає кастомний `DataImportError`.

Знайдені потенційні defects

У процесі написання тестів та розробки коду було попереджено такі потенційні дефекти:

1. Запис некоректних або від'ємних оцінок (заблоковано у `__post_init__`).
2. Падіння програми через ділення на нуль при запиті середнього бала порожнього журналу (додано умову `if not self.records: return 0.0`).
3. Витік мережевих помилок як невідомих винятків замість опрацювання на рівні сервісу.
4. Пропуск пошкоджених рядків у CSV, які могли б тихо спотворити фінальну статистику (замінено на жорсткий `raise DataImportError`).

Відповіді на контрольні запитання

1. **Automated test** — програмний код, який запускає певну частину системи та автоматично звіряє її результат із очікуваним.
2. **Переваги тестування** — автоматичний пошук дефектів, захист від регресій, можливість безпечного рефакторингу, жива документація коду.
3. **Regression** — ситуація, коли після змін у коді знову з'являється стара помилка або ламається функціонал, який раніше працював.
4. **Unit test** — швидка та ізольована перевірка одного мінімального компонента системи (наприклад, окремого класу чи функції).
5. **Integration test** — перевірка того, як кілька реальних компонентів або систем працюють разом.
6. **Test pyramid** — концепція, згідно з якою основу тестування мають складати швидкі та дешеві unit-тести (багато), вище — integration-тести (менше), а на вершині — end-to-end тести (найменше).
7. **pytest** — популярний і потужний фреймворк для тестування у Python.
8. **Як pytest знаходить tests?** — Автоматично шукає файли з префіксом `test_` або суфіксом `_test.py`, а в них — функції, що починаються на `test_`.
9. **Основна команда запуску** — `pytest`.

10. **Для чого pytest -v і -x?** — `-v` (verbose) виводить детальний список пройдених тестів, а `-x` зупиняє виконання одразу після першого ж проваленого тесту.
11. **Arrange-Act-Assert** — шаблон написання тесту: підготувати дані (Arrange), виконати дію (Act), перевірити результат (Assert).
12. **assert** — ключове слово Python для перевірки істинності виразу. Якщо вираз `False`, генерується `AssertionError`.
13. **Assertion failure** — помилка, що виникає, коли фактичний результат роботи коду не збігається з очікуваним у `assert`.
14. **Floats не завжди слід порівнювати через ==**, оскільки через особливості двійкової арифметики `0.1 + 0.2` може не дорівнювати точно `0.3`. Тому використовують `pytest.approx`.
15. **pytest.raises** — контекстний менеджер для перевірки того, що код навмисно генерує правильний виняток у відповідь на некоректні дії.
16. **Exception object** отримують за допомогою конструкції `with pytest.raises(ValueError) as exc_info:`, щоб потім перевірити деталі об'єкта `exc_info.value`.
17. **Boundary testing** — техніка тестування на межах допустимих діапазонів (там найчастіше ховаються баги, наприклад, помилки >= проти >).
18. **Для діапазону 0-100** слід тестувати: від'ємні числа (-0.1, -1), точний 0, типові значення всередині (50), точні 100 та числа поза межами (100.1, 150).
19. **Parametrization** — механізм запуску однієї тестової функції багато разів з різними аргументами.
20. **ids** у параметризації дозволяють дати кожному набору даних зрозумілу назву, яка відображатиметься у звітах (наприклад, `id="negative_grade"`).
21. **Fixture** — функція, яка готує стан або дані (setup) перед запуском тесту.
22. **yield у fixture** розділяє логіку: код до `yield` виконує підготовку, а код після `yield` — очищення (teardown) після завершення тесту.
23. **Fixture scopes:** function (за замовчуванням, створюється для кожного тесту), class, module, package, session.
24. **conftest.py** — спеціальний файл, де зберігаються спільні фікстури, які стають автоматично доступними для всіх тестів у директорії без необхідності їх імпортувати.
25. **tmp_path** — вбудована фікстура pytest, яка надає унікальний тимчасовий каталог (`pathlib.Path`) для безпечного створення файлів під час тестів.
26. **Test isolation** — принцип, за яким результат одного тесту не повинен залежати від порядку запуску чи залишків даних інших тестів.
27. **monkeypatch** — фікстура для тимчасової безпечної зміни глобального стану (змінних середовища, функцій) на час виконання одного тесту.

28. **Environment variable** змінюють через `monkeypatch.setenv("KEY", "value")`, а видаляють через `monkeypatch.delenv("KEY")`.
29. **Test double** — загальний термін для об'єктів-замінників (заглушок), які використовуються у тестах замість реальних залежностей.
30. **Dummy** — порожній об'єкт, який передається лише для задоволення сигнатури функції, але його методи не викликаються.
31. **Stub** — об'єкт, який завжди повертає жорстко зашиті (заздалегідь підготовлені) відповіді.
32. **Fake** — об'єкт зі спрощеною робочою логікою (наприклад, In-Memory словник замість реальної SQL бази даних).
33. **Mock** — об'єкт, який імітує поведінку залежності та запам'ятовує всі виклики до нього (з якими аргументами і скільки разів він викликався).
34. **Spy** — об'єкт, який обгортає реальну реалізацію: дозволяє їй працювати нормально, але таємно записує історію її викликів.
35. **spec** обмежує Mock так, щоб він мав лише ті методи, які існують у реального класу. Це запобігає виклику неіснуючих методів на заглушці.
36. **return_value** — атрибут Mock, що дозволяє задати значення, яке Mock поверне при виклику без генерування помилок.
37. **side_effect** — атрибут Mock, за допомогою якого можна змусити Mock згенерувати виняток (`side_effect = ValueError`) або виконувати кастомну функцію.
38. **Один виклик** перевіряється методами `assert_called_once()` або `assert_called_once_with(args)`.
39. **Відсутність виклику** перевіряється методом `assert_not_called()`.
40. **State testing** — перевірка того, як змінився стан об'єкта чи системи після виконання дії.
41. **Interaction testing** — перевірка того, чи правильно тестований об'єкт викликав методи своїх залежностей (чи надіслав повідомлення, чи зберіг у БД).
42. **Interaction testing виправдане** тоді, коли зміна стану невидима ззовні, або коли виклик зовнішньої системи (наприклад, відправка Email) є основною метою функції.
43. **patch** — інструмент (часто декоратор), що замінює реальний об'єкт моком. Важливо патчити об'єкт у тому просторі імен, де він використовується, а не де він визначений.
44. **AsyncMock** — спеціальний мок для імітації асинхронних функцій та корутин (ті, що очікуються через `await`).
45. **Awaited call** перевіряється методом `assert_awaited_once_with()`.

46. **Flaky test** ("мерехтливий" тест) — тест, який може то проходити успішно, то падати без жодних змін у коді (наприклад, через залежність від часу чи мережі).
47. **Чому flaky tests небезпечні?** — Вони змушують розробників ігнорувати результати тестування, нівелюючи весь сенс процесу CI/CD.
48. **Deterministic test** — тест, результат якого на 100% передбачуваний і однаковий за умови незмінності коду.
49. **Unit test не повинен залежати від Internet**, оскільки це робить його повільним, недетермінованим і таким, що падає при відсутності з'єднання.
50. **Real filesystem доречний** в integration-тестах для перевірки реального парсингу та серіалізації, але лише з використанням `tmp_path` для ізоляції.
51. **Не слід mock-ати все**, бо тоді тест перетворюється на перевірку самої конфігурації моків, а реальна взаємодія класів залишається неперевіреною.
52. **Over-mocking** (надмірне мокування) призводить до крихких тестів, які ламаються при найменшому рефакторингу, навіть якщо загальна логіка системи правильна.
53. **Coverage** (покриття коду) — метрика, що вказує відсоток вихідного коду, який був виконаний хоча б раз під час прогону тестів.
54. **Statement coverage** — відсоток виконаних рядків (операторів) коду.
55. **Branch coverage** — відсоток виконаних гілок (branch) в умовних операторах (`if/else`, `for`, `while`, `except`). Це набагато суворіша метрика, ніж statement coverage.
56. **Coverage запускається** командою `pytest --cov=module_name`.
57. **--cov-report term-missing** — прапорець, який змушує pytest виводити безпосередньо в консоль номери рядків коду, які тести не виконали.
58. **HTML report** створюється для зручної візуалізації: він підсвічує зеленим/червоним кольором безпосередньо у браузері рядки, які покриті чи пропущені.
59. **100% coverage не гарантує** відсутності багів, оскільки тест може просто викликати функцію і пройти весь код, але не містити жодного `assert` для перевірки результату.
60. **Meaningful assertions важливіші**, оскільки саме вони захищають логіку програми від помилок і регресій. Покриття без перевірок (asserts) — це просто ілюзія безпеки.
61. **Coverage threshold** (поріг покриття) — налаштування (наприклад, у `pyproject.toml fail_under = 80`), яке автоматично провалює тестовий запуск, якщо покриття падає нижче заданої позначки.

62. **Tests повинні бути незалежними**, щоб їх можна було запускати по одному для відлагодження, і щоб можна було запускати їх паралельно для прискорення. Залежні тести — це антипатерн.

Висновок

У межах виконання лабораторної роботи розроблено повноцінний автоматизований test suite для системи оцінювання студентів. На практиці закріплено використання фреймворку `pytest`. Реалізовано unit та integration tests, які покривають нормальні (happy path), межові (boundary), невалідні та аварійні (failure) сценарії. Застосування `pytest.mark.parametrize` та `fixtures` значно зменшило дублювання коду, а `pytest.raises` забезпечив чітку перевірку виняткових ситуацій. Взаємодія з файловою системою була надійно ізольована завдяки фікстурі `tmp_path`, конфігурація перевірялася через `monkeypatch`, а зовнішні та асинхронні залежності успішно підмінялися через `Mock` та `AsyncMock`. Наскрізні інтеграційні тести підтвердили працездатність повного потоку: від імпорту та фільтрації до експорту. Контроль якості коду забезпечено завдяки аналізу statement та branch coverage, які досягли показника у 100%. Підтверджено, що ізольоване тестування зі змістовними перевірками (`assertions`) є критично важливим етапом створення надійного програмного забезпечення.